

Student Instructions: Building the Shared Provenance Graph

CausalGuard Foundation Prototype

0. Background: Why We Are Building This

CausalGuard is a security framework for agentic AI systems. These systems do not merely answer questions. They call tools, read data, invoke APIs, trigger actions, and make multi-step decisions. A single operation may be allowed on its own, while the larger chain of operations violates a security policy.

The shared provenance graph is the foundation that lets us reason about those chains. It records what happened, what caused what, what data was touched, what action followed, and whether the action had the right authorization. Without this graph, we only have isolated logs. With this graph, we can ask causal security questions.

Example: an agent may be allowed to read a contacts file and may also be allowed to send a network request. But it should not send contacts to an untrusted destination without action-specific approval. That violation appears only when we connect the causal chain:

```
user input -> LLM invocation -> tool call -> file read -> contacts.csv -> network send -> external destination
```

Your task is to build the first clean version of this shared graph foundation. This version should not focus on optimization. It should focus on correctness, clarity, privacy, and testability.

1. Project Goal

Build a compact, typed, privacy-preserving provenance DAG that records agentic AI execution events and causal relationships among them.

The prototype should support questions such as:

- Which LLM invocation caused this tool call?
- Which system operations did the tool trigger?
- Which data objects were accessed?
- Did a network send involve sensitive data?
- Was there valid, action-specific human approval?

The first prototype can use synthetic traces and a simple mock agent workflow. Real agent-framework integration can come later.

2. Scope of This First Version

Build only the graph foundation. Do not build a high-performance runtime detector yet. Do not build graph indices yet. Do not optimize for latency, memory, storage, or mobile deployment. Those concerns will matter later, after the graph semantics are correct.

The first version should include:

1. A normalized event schema.
2. Typed node and edge schemas.
3. An in-memory graph store.
4. A graph builder that converts event traces into a DAG.
5. Basic graph queries for debugging and validation.
6. Synthetic traces and unit tests.
7. A graph visualization export.

3. Design Principles

3.1 Preserve causality

Every edge should mean something causal or authorizing. Avoid vague links such as “related_to.” If the graph says A caused B, the implementation should record why that edge exists.

3.2 Keep the graph typed

Do not store everything as a generic event. The graph must distinguish model calls, tool calls, system operations, data objects, and approvals. Security policies depend on these distinctions.

3.3 Avoid raw sensitive content

The graph should not store raw prompts, raw model outputs, raw files, contacts, credentials, health data, or user messages. Use hashes, stable IDs, normalized summaries, labels, and redacted metadata.

3.4 Keep uncertainty explicit

Some causal links will be explicit, such as a parent event ID. Others may be inferred from a shared context or a short time window. Record the derivation and confidence for each edge.

3.5 Keep the first version simple

Prefer a small, correct implementation over a broad but fragile one. Build a foundation another student can understand, test, and extend.

4. Recommended Repository Structure

```
causalguard-graph/  
  README.md  
  pyproject.toml  
  causalguard/  
    __init__.py  
    schema/
```

```

__init__.py
events.py
nodes.py
edges.py
graph/
  __init__.py
  store.py
  builder.py
  queries.py
collectors/
  __init__.py
  mock_collector.py
examples/
  contacts_upload_trace.jsonl
  approval_trace.jsonl
  device_action_trace.jsonl
tests/
  test_schema.py
  test_graph_store.py
  test_graph_builder.py
  test_queries.py
scripts/
  build_graph_from_trace.py
  print_graph.py
  export_graph.py

```

Use Python for the first prototype. Recommended libraries: pydantic for validation, networkx for the in-memory DAG, pytest for tests, and orjson or json for trace loading.

5. Step-by-Step Instructions

Step 1: Define the Normalized Event Schema

Every collector should eventually emit a common event format. For now, synthetic JSONL traces can use this schema.

```

event_id: str
timestamp: float
event_type: str
agent_id: str | None
session_id: str | None
causal_context_id: str | None
parent_event_id: str | None
attributes: dict

```

Start with these event types:

```
user_input
llm_invocation
tool_call
system_operation
data_access
human_approval
network_send
device_command
```

The two most important relationship fields are `parent_event_id` and `causal_context_id`. The `parent_event_id` gives a direct causal link. The `causal_context_id` groups events from the same execution context.

Deliverable: `causalguard/schema/events.py` with validation tests.

Step 2: Define the Typed Node Schema

Implement five node types.

LLM Invocation Node

```
node_id: str
node_type: "llm_invocation"
model_name: str | None
prompt_hash: str | None
output_hash: str | None
token_count: int | None
timestamp: float
agent_id: str | None
session_id: str | None
causal_context_id: str | None
```

Tool Call Node

```
node_id: str
node_type: "tool_call"
tool_name: str
action_class: str | None
argument_summary: dict
target_resource: str | None
destination: str | None
timestamp: float
agent_id: str | None
session_id: str | None
causal_context_id: str | None
```

Examples of `action_class`: `read_contacts`, `send_network`, `make_payment`, `unlock_door`, `export_calendar`, `read_health_data`.

System Operation Node

```
node_id: str
node_type: "system_operation"
operation_type: str
syscall_kind: str | None
resource_id: str | None
destination: str | None
byte_count: int | None
timestamp: float
agent_id: str | None
session_id: str | None
causal_context_id: str | None
confidence: str
```

Examples of operation_type: file_read, file_write, network_send, network_connect, ipc_call, device_command, payment_action, cloud_api_call.

Data Object Node

```
node_id: str
node_type: "data_object"
resource_id: str
object_kind: str
content_hash: str | None
sensitivity: str
trust_label: str | None
owner: str | None
```

Examples of sensitivity: public, internal, PII, credential, health, payment, location, unknown. Examples of trust_label: trusted, untrusted, external, user_approved, unknown.

Human Approval Node

```
node_id: str
node_type: "human_approval"
approver_id: str
action_class: str
resource_scope: str | None
destination_scope: str | None
timestamp: float
expiration: float | None
session_id: str | None
causal_context_id: str | None
```

Approval must be scope-sensitive. Approval for one action, resource, destination, or session should not silently authorize another.

Deliverable: causalguard/schema/nodes.py with validation tests.

Step 3: Define the Edge Schema

Implement four edge types.

Edge type	Meaning	Example
invokes	An LLM invocation caused a tool call.	LLM -> share_contacts tool
triggers	A tool call caused a system operation.	share_contacts -> file_read
accesses	A system operation touched a data object.	file_read -> contacts.csv
authorizes	A human approval authorized an action or scope.	approval -> upload contacts.csv

Each edge should include:

```
edge_id: str
source_id: str
target_id: str
edge_type: str
timestamp: float
derivation: str
confidence: str
evidence_ref: str | None
```

Examples of derivation: parent_event, causal_context, temporal_window, explicit_approval_scope.

Deliverable: causalguard/schema/edges.py with validation tests.

Step 4: Implement the In-Memory Graph Store

Use networkx.DiGraph for the first version. The graph store should expose a small API:

```
add_node(node)
add_edge(edge)
get_node(node_id)
get_successors(node_id, edge_type=None)
get_predecessors(node_id, edge_type=None)
find_nodes(node_type=None, **filters)
export_json()
export_dot()
```

Requirements:

- 8. Reject duplicate node IDs unless explicitly allowed.
- 9. Reject edges whose source or target node does not exist.
- 10. Reject edges that create a cycle.
- 11. Store node and edge metadata.
- 12. Export JSON for debugging.
- 13. Export DOT or Mermaid for visualization.

Deliverable: causalguard/graph/store.py.

Step 5: Implement the Graph Builder

The graph builder consumes normalized events and updates the graph.

```
class GraphBuilder:
    def __init__(self, graph_store, config):
        ...

    def process_event(self, event):
        ...

    def process_trace(self, events):
        ...
```

The builder has four responsibilities:

14. Convert events into typed nodes.
15. Insert nodes into the graph.
16. Create causal or authorization edges when possible.
17. Record edge derivation and confidence.

Start with deterministic conversion rules:

```
llm_invocation event -> LLM node
tool_call event -> tool-call node
system_operation event -> system-operation node
data_access event -> system-operation node + data-object node
network_send event -> system-operation node + destination data-object node
human_approval event -> human-approval node
device_command event -> system-operation node
```

Then add edge construction rules:

```
If a tool_call has parent_event_id pointing to an llm_invocation:
    add an invokes edge.

If a system_operation has parent_event_id pointing to a tool_call:
    add a triggers edge.

If a system_operation touches resource_id or destination:
    add an accesses edge to the corresponding data_object.

If a human_approval scope matches an action/resource/destination:
    add an authorizes edge.

If explicit parent links are missing but events share causal_context_id:
    add a lower-confidence edge with derivation="causal_context".
```

If both parent links and context are missing, optionally use a short time window:

add a low-confidence edge with derivation="temporal_window".

Deliverable: causalguard/graph/builder.py.

Step 6: Build a Mock Collector and Example Traces

Before integrating any real framework, build a mock collector that reads JSONL traces. Each line should be one normalized event.

Example contacts-upload trace:

```
{
  "event_id": "e1",
  "timestamp": 1.0,
  "event_type": "user_input",
  "agent_id": "agent1",
  "session_id": "s1",
  "causal_context_id": "ctx17",
  "parent_event_id": null,
  "attributes": {
    "input_hash": "sha256:user-request"
  }
}

{
  "event_id": "e2",
  "timestamp": 2.0,
  "event_type": "llm_invocation",
  "agent_id": "agent1",
  "session_id": "s1",
  "causal_context_id": "ctx17",
  "parent_event_id": "e1",
  "attributes": {
    "model_name": "gpt-4o",
    "prompt_hash": "sha256:p",
    "output_hash": "sha256:o",
    "token_count": 120
  }
}

{
  "event_id": "e3",
  "timestamp": 3.0,
  "event_type": "tool_call",
  "agent_id": "agent1",
  "session_id": "s1",
  "causal_context_id": "ctx17",
  "parent_event_id": "e2",
  "attributes": {
    "tool_name": "share_contacts",
    "action_class": "upload",
    "target_resource": "contacts.csv",
    "destination": "external.example"
  }
}

{
  "event_id": "e4",
  "timestamp": 4.0,
  "event_type": "data_access",
  "agent_id": "agent1",
  "session_id": "s1",
  "causal_context_id": "ctx17",
  "parent_event_id": "e3",
  "attributes": {
    "operation_type": "file_read",
    "resource_id": "contacts.csv",
    "object_kind": "file",
    "sensitivity": "PII"
  }
}

{
  "event_id": "e5",
  "timestamp": 5.0,
  "event_type": "network_send",
  "agent_id": "agent1",
  "session_id": "s1",
  "causal_context_id": "ctx17",
  "parent_event_id": "e3",
  "attributes": {
    "destination": "external.example",
    "trust_label": "untrusted",
    "byte_count": 2048
  }
}
```

After processing this trace, the graph should contain an LLM node, a tool-call node, system-operation nodes for file read and network send, data-object nodes for contacts.csv and external.example, and invokes, triggers, and accesses edges.

Deliverables: causalguard/collectors/mock_collector.py and examples/contacts_upload_trace.jsonl.

Step 7: Implement Basic Graph Queries

This is not the final policy engine. These queries are for debugging, validation, and early demonstrations.

```
get_context_subgraph(causal_context_id)
get_upstream_causes(node_id, max_depth=5)
get_downstream_effects(node_id, max_depth=5)
find_sensitive_data_accesses(context_id)
find_untrusted_network_sends(context_id)
find_valid_approval(action_class, resource, destination, session_id, now)
```

Then implement one smoke-test query:

```
detect_pii_upload_without_approval(context_id, now)
```

It should return a structured result such as:

```
{
  "violation": true,
  "policy": "pii_upload_without_approval",
  "sensitive_resource": "contacts.csv",
```



```

    "destination": "external.example",
    "approval_found": false,
    "evidence_nodes": [...],
    "evidence_edges": [...]
}

```

Deliverable: causalguard/graph/queries.py.

Step 8: Add Graph Visualization

Create a script that exports the graph to DOT or Mermaid. The output should make causal structure easy to inspect.

```

python scripts/build_graph_from_trace.py examples/contacts_upload_trace.jsonl --export-dot
contacts.dot

```

Use compact labels such as LLM, Tool, SysOp, Data, and Approval. Do not print raw content.

Deliverables: scripts/export_graph.py and scripts/print_graph.py.

Step 9: Add Unit Tests

Add tests for the foundation. Minimum required tests:

Schema tests

18. Valid event passes validation.
19. Missing required field fails validation.
20. Invalid node type fails validation.
21. Invalid edge type fails validation.

Graph store tests

22. Add node.
23. Add edge.
24. Reject edge with missing endpoint.
25. Reject cycle.
26. Export graph.

Graph builder tests

27. LLM event creates LLM node.
28. Tool event creates tool node.
29. Data access creates system-operation and data-object nodes.
30. Network send creates system-operation and destination object nodes.
31. Parent event creates the correct edge.
32. Shared causal context creates a lower-confidence edge when no parent link exists.
33. Temporal fallback, if implemented, creates a low-confidence edge.

End-to-end test

34. Process contacts_upload_trace.jsonl.
35. Confirm the graph is acyclic.
36. Confirm expected node and edge types exist.
37. Confirm the PII-upload query returns a violation when no approval exists.

38. Add a valid approval event and confirm the query no longer reports a violation.

6. Concrete Demo Target

The first demo should show this story:

```
A user asks the agent to share contacts.  
The LLM invokes a share_contacts tool.  
The tool reads contacts.csv.  
The tool sends data to external.example.  
No valid approval exists.
```

The resulting graph should be:

```
LLM -> Tool -> File read -> contacts.csv  
-> Network send -> external.example
```

The query result should be:

```
Violation: PII upload to untrusted destination without approval.
```

Then add this event:

```
User approves uploading contacts.csv to external.example during the current session.
```

The updated query result should be:

```
No violation because exact valid approval exists.
```

7. Suggested Timeline

Week	Focus	Expected output
1	Repository, event schema, node schema, edge schema	Validated schema classes and tests
2	In-memory graph store	Working typed DAG with export support
3	Graph builder	Synthetic traces converted into graph nodes and edges
4	Queries and example traces	End-to-end contacts-upload demo
5	Visualization and README	Inspectable graph and runnable instructions
6	Hardening and review	Stable prototype with tests and design note

8. Weekly Reporting Template

Each weekly update should include:

39. What was implemented.
40. What tests pass.
41. What design decisions were made.
42. What assumptions are currently hard-coded.

43. What examples currently work.
44. What is blocked or unclear.
45. What you plan to do next.

Keep updates short and concrete. Include commands, test output, and graph screenshots when useful.

9. Final Deliverables

At the end of this task, provide:

46. A Git repository with the graph prototype.
47. A README with setup and demo instructions.
48. Event, node, and edge schema definitions.
49. Graph store implementation.
50. Graph builder implementation.
51. Basic graph query functions.
52. Synthetic traces.
53. Unit tests.
54. One visualization of the contacts-upload graph.
55. A short design note explaining assumptions, limitations, and next steps.

The design note should answer:

```
What events do we capture?  
What nodes do we create?  
What edges do we create?  
How do we link events causally?  
How do we avoid storing sensitive raw content?  
How do we represent uncertainty?  
What queries can the graph already support?  
What must be added before integration with a real agent runtime?
```

10. Future Work: Leave Optimization for Later

Do not implement graph indices in this first version. Do not spend time on storage compression, low-latency matching, incremental policy optimization, mobile overhead, or production telemetry adapters. These are important, but they should come after the foundation is correct.

Possible future extensions include:

- Runtime indices for context, resources, approvals, and time windows.
- Graph compaction for repeated low-level operations.
- A real agent-framework collector.
- OpenTelemetry trace/span import.
- Bounded backward slicing for root-cause attribution.
- A policy-template interface.

- Benchmark traces for prompt injection, credential leakage, approval misuse, payment misuse, and unsafe device commands.

11. Summary

The shared graph is the foundation of CausalGuard. Build it as a compact, typed, privacy-preserving causal DAG. It should connect agent decisions to tool calls, data accesses, approvals, and system effects. It should be easy to inspect, easy to test, and easy to extend.

The first milestone is simple but important: given a trace of an agent reading contacts and sending them to an untrusted destination, the graph should reconstruct the causal chain and show whether exact user approval exists.